# INDEX

23MIS0403

NAME: S. Sharusree STD.: ______ SEC.: ______ ROLL NO.: ______ SUB.: Artificial Intelligence LAB

| S. No. | Date | Title | Page No. | Teacher's Sign / Remarks |
|---|---|---|---|---|
| | | | | |
| Exercise1 | 9.01.2026 | Breadth First Search | 2-5 | A.A7 |
| Exercise2 | 9.01.2026 | Depth First Search | 6-9 | A.A7 |
| Exercise 3 | 23.01.2026 | Iterative Deepening Search | 10-13 | A.A7 24/01/26 |
| Exercise4 | 23.01.2026 | Depth Limited search | 14-17 | |
| Exercise5 | 23.01.2026 | Uniform Cost Search | 18-21 | |
| Exercise:6 | 13.02.26 | greedy | 22-25 | A.A7 13/02/26 |
| Exercise:7 | 13.02.26 | a* star | 25-28 | |
| Exercise:8 | 13.02.26 | TSP | 29-32 | |
| Exercise:9 | 13.02.26 | minimax | 33-34 | |
| Exercise:10 | 13.02.26 | alphabeta | 35-38 | |
| Exercise:11 | 13.03.26 | Linear Regression | | A.A7 27/03/2026 |
| Exercise:12 | 13.03.26 | Logiestic Regression | | |
| Exercise:13 | 13.03.26 | k-means Clustering | | |
| Exercise:14 | 13.03.26 | Decision Tree Classifiers. | | |
| Exercise:15 | 13.03.26 | Random Forest Regression | | |

# Exercise - 3

## ASSESSMENT - 03

①. Consider the following small dataset on housing prices applying linear Regression algorithm to predict a continous value based on feateous in the dataset.

| (size) Style | Bedrooms | Price |
|---|---|---|
| 1500 | 3 | 300000 |
| 2000 | 4 | 400000 |
| 2500 | 4 | 500000 |
| 1800 | 3 | 350000 |
| 2200 | 4 | 450000 |
| 1700 | 3 | 325000 |
| 2100 | 4 | 420000 |
| 1600 | 3 | 310000 |
| 2400 | 5 | 490000 |
| 1900 | 3 | 370000 |

Instructions

- Install required libraries → pip install Panda as
- store the alone dataset in housing pricesx Ls x and place it in the same as year python script.
- Run the python code
  - i) To Load the execel file
  - ii) preprocess the data
  - iv) ~~Display~~ the mean squares error and a plot comparing actual vs predicated housing prices.
  - iii) Trains a linear regression model.

SOURCE_CODE :

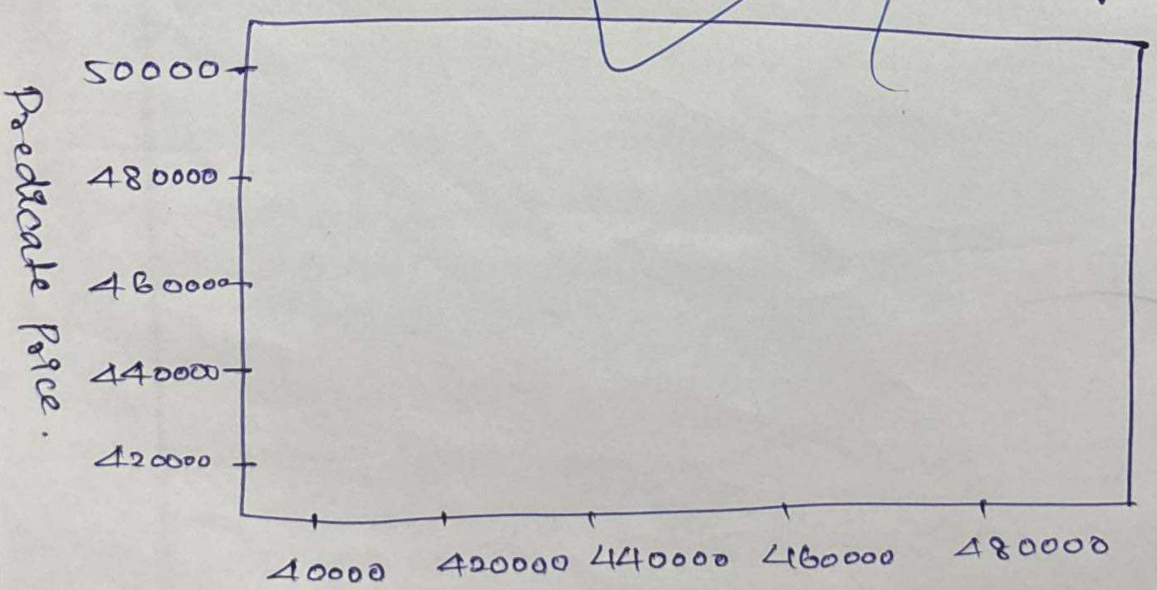
```
import panda as pd
import matplotlib. pyplot as plt
from sklearn. model selection import train_
                                test_ split.
from sklearn. linear_model import linear
                                Regression.
from sklearn. metrics import mean_squard_error
data = pd. read_excel ("housing_prices. xlsx")
print ("Dataset :")
print ( data)
x = data [['size', 'bedrooms']]
y = data ['price']
x_train, x _ test, y_train, y_test = train_test
      _ split (x, y, test size = 0.3 random_
                                State = 42)
model = linear Regression ()
model. fit (x_train, y_train)
y_pered = model. predict (x_test)
mse = mean_squared_error (y_test, y_pred)
print ("In mean squared Error: ", mse)
plt. scatter (y_test, y_pred)
plt. xlabel ("Actual prices")
plt. ylabel ("Predicted Prices")
plt. title ("Actual vs Predicted Housing Prices")
Plt. show()
```

OUTPUT :

| Size | Bedrooms | Price |
|---|---|---|
| 1500 | 3 | 300000 |
| 2000 | 4 | 400000 |
| 2500 | 4 | 500000 |
| 1800 | 3 | 350000 |
| 2200 | 4 | 450000 |
| 1700 | 3 | 325000 |
| 2100 | 4 | 420000 |
| 1600 | 3 | 310000 |
| 2400 | 5 | 490000 |
| 1900 | 3 | 370000 |

Mean Squared Error : 95806122 . 4489 7911

Actual vs Predicted Housing Prices



(2). Considering the following small dataset on spam classification, classify instances into binary categories (e.g: spam or not spam) using Logistic Regression.

| Email Length | Has_link | In spam(0 or 1) |
|---|---|---|
| 50 | 1 | 0 |
| 300 | 1 | 1 |
| 200 | 0 | 0 |
| 400 | 1 | 1 |
| 150 | 0 | 0 |
| 350 | 1 | 1 |
| 100 | 0 | 0 |
| 500 | 1 | 1 |
| 250 | 0 | 0 |
| 450 | 1 | 1 |

Instructions:

- Load the Excel file using pandas
- Perform data preprossing (e.g: encode categories Variables)
- Split the data into training & testing sets.
- Train a Logistic Regression model.
- Evaluate the model with accuracy, Precision, recall.

SOURCE - CODE :

```
import pandas as pd
df = pd.read_excel("spam-dataset.
                         xlse")
print (df.head())
x= df [["Email-length","Has-link"]]
y = df ["Is-spam"]
from sklearn.model_selection import train_
                         test_split.
x_train, x_test, y-train, y_test = train_test_
              split (x, y, test_size =0.3,
                         random_state =42)
from sklearn.linear_model" import logistic
                         Regression.
model = Logistic Regression ()
model.fit (x_train, y_train)
y_pred = model.predict (x_test)
from sklearn.metrics import accuracy_score,
    presion_score, recall_score, confusion_matrix
accuracy = accuracy_score (y_test, y_pred)
precision = precision_score (y_test, y_pred)
recall = recall_score (y_test, y_pred)
cm = Confusion_matrix (y_test, y_pred)
print ("Accuracy:", accuracy)
print (" Precision:", precision)
print (" Recall :", recall)
print (" Confusion matrix :\n", cm)
```

Output :

Accuracy : 0.66666666

Precisions : 1.0

Recall : 0.5

Confusion matrex : [[1, 0][1, 1]]

or

```
[ [1, 0]
  [1, 1]]
```

③. Consider the following small dataset, Group data points into clusters based on their features using k-means clustering.

| Income | Age | Spending score |
|---|---|---|
| 40000 | 22 | 65 |
| 50000 | 25 | 55 |
| 60000 | 30 | 50 |
| 70000 | 35 | 40 |
| 80000 | 40 | 45 |
| 30000 | 20 | 70 |
| 90000 | 45 | 30 |
| 100000 | 50 | 20 |
| 110000 | 55 | 15 |
| 120000 | 60 | 10 |

Instructions:

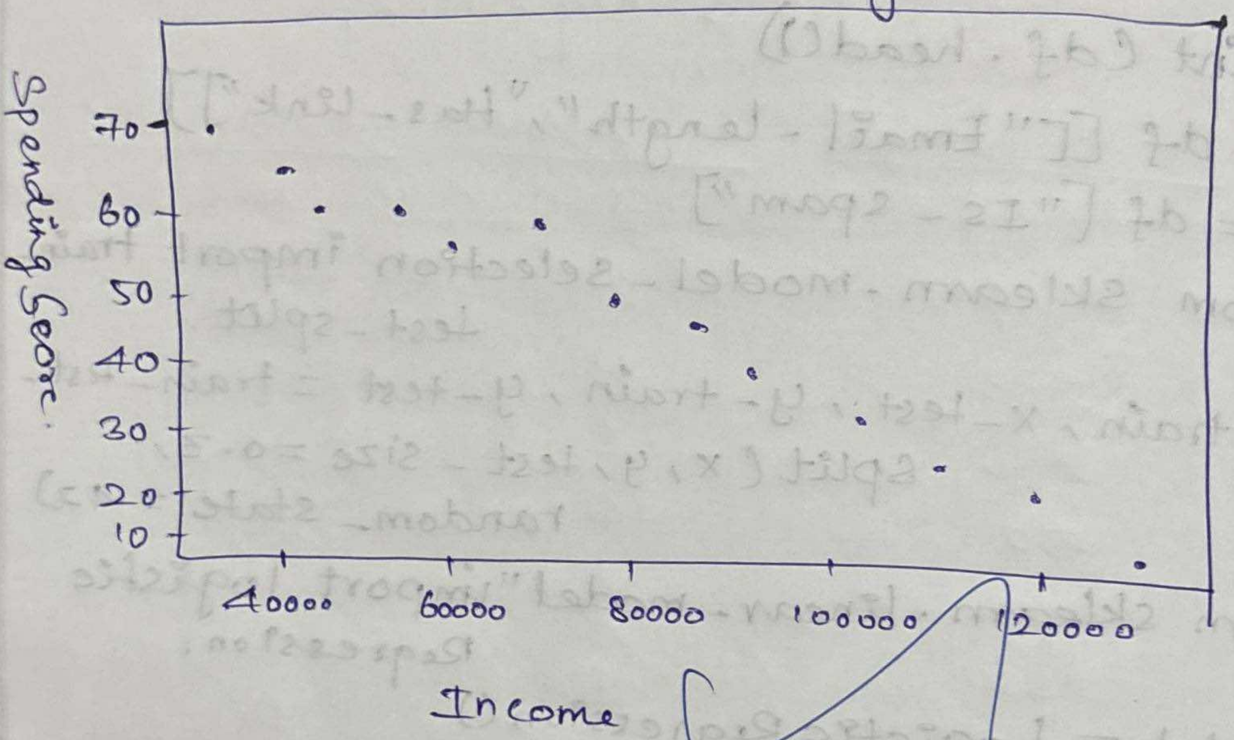
* Load the Excel file using pandas.
* Preprocess the data [e.g., scaling features]
* Apply k-means clustering to identify clusters in the dataset.
* Visualize the clusters using a scatter plot.

SOURCE CODE:

```
import pandb as pd
import warnings
warnings.filter warnings(" ignore ")
df = pd.read_excel ("Customer_data.xlsx")
print (df.head())
from sklearn.preprocessing import standard
                                      Scaler
Scaler = standard Scaler()
x_saled = scaler.file_transtorm (df)
from sklearn.cluster import kmeans.
kmeans = kmeans(n_cluster = 3, random_state=
                                      42)
kmeans.fit (x_scaled)
df["cluster"] = kmeans.labels_ ~~print~~
print(df)
import matplotlib.pyplot as plt
plt.scalter(df["Income"], df["spending-
     Score"], c=df["Cluster"], cmap= "viridis")
plt.xlabel ("Income")
plt.ylabel ("Spending Score")
plt.title(" k-means clustering")
plt.show()
```

OUTPUT:

K-means clustering



4. Considering the following small dataset, classify instances into multiple categories. (e.g., species of flowers) using Decision Tree Classifier.

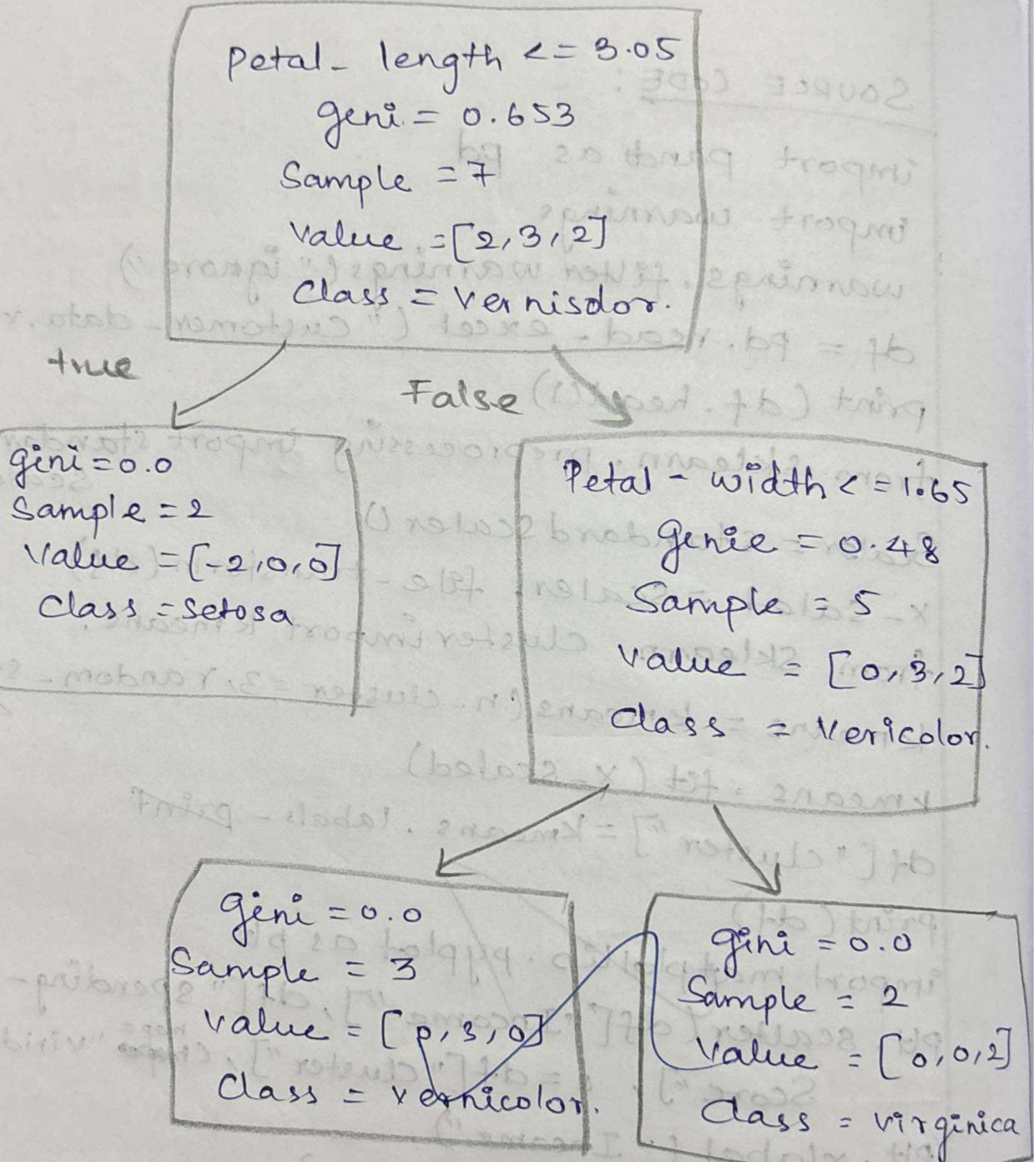
| Petal Length | Petal width | Species |
|---|---|---|
| 1.4 | 0.2 | Santosa |
| 1.3 | 0.2 | santosa |
| 4.7 | 1.4 | vernicolor |
| 4.5 | 1.5 | Vernicolor |
| 5.5 | 2.1 | verginica |
| 5.1 | 1.9 | virginica |
| 4.9 | 1.8 | virginica |
| 1.6 | 0.4 | Setosa |
| 4.2 | 1.3 | vernicolor |
| 0 | 1.5 | vernicolor. |

Introduction:

- Load the Excel file using pandas
- Prepurpose the data
- split the data into training & testing sets.
- Trains a Decision Tree classifier
- Visualize the true & Evaluate the models accuracy.

SOURCE CODE:

```
import Pandas as pd
df = pd.read_excel ["flower.xlsx", header=1]
print (df.head(1)
from sklearn. praprocessing import Label Encoder
le = Label Encoder()
df ["species_encoded"] - Le.fit_transform
                        (af ["species"])
print(def)
from sklearn.model_selection import.train_
                                test_split
x = af [["petal_length", "petal_width"]]
y = df ["species_encoded"]
x_train, x_test, y_train, y_test = train_test_
        split (x, y, test_size = 0.3, random_
                        state = 42)
from sklearn, tree import Decision Tree Classifier
model = Decision Tree Classifier (random_states
                                        =42)
Model.fit (x_train, x_train)
from sklearn.metrices import accurancy_
                                store
y_pred = model. predict (x_test)
accurany = accuracy: ", accuracy)
print (" Accuracy:")
import matplotlib. pyplot = true.
from Sklearn.tree import plot_tree
plt. tree (model, feature_names = ["petal_length",
                        "petal_width"],
                class_names = le.classes,
                        fluid = TRUE)
pt. show ()
```


Petal_ length <= 3.05
geni = 0.653
Sample = 7
Value = [2, 3, 2]
Class = Vernisdor.
true
False
gini = 0.0
sample = 2
Value = [-2, 0, 0]
Class = setosa
Petal - width <= 1.65
Genie = 0.48
Sample = 5
Value = [0, 3, 2]
Class = Vericolor.
gini = 0.0
Sample = 3
Value = [0, 3, 0]
Class = vernicolor.
gini = 0.0
Sample = 2
Value = [0, 0, 2]
Class = virginica

output:

| petal Length | petal_width | Species | Species_encoded |
| --- | --- | --- | --- |
| 1.4 | 0.2 | Setosa | 0 |
| 1.3 | 0.2 | Setosa | 0 |
| 4.7 | 1.5 | Vernicolor | 1 |
| 4.5 | 2.7 | vernicolor | 1 |
| 5.5 | 1.9 | virginica | 2 |
| 5.1 | 1.8 | virginica | 2 |
| 4.9 | 0.4 | ~~Setosa~~ virginia | 0 |
| 1.6 | 1.3 | Setosa | 0 |
| 4.2 | 1.5 | Vernicolor | 1 |
| 5.0 | 1.5 | Vernicolor | 1 |

⑤. Considering the following small dataset, predict a continous value using an ensemble of decision trees using random Forest Regression.

| Data | (°C) Temperature | (%) Humidity | Energy Consumption |
|---|---|---|---|
| 01-09-2024 | 22 | 60 | 30 |
| 02-09-2024 | 24 | 55 | 32 |
| 03-09-2024 | 23 | 58 | 31 |
| 04-09-2024 | 25 | 50 | 29 |
| 05-09-2024 | 26 | 52 | 34 |

Introduction:

- Load the Excel file using pandas
- preprocess the data.
- split the data into training & testing part / model.
- Train Random Forest model.
- Evaluate the model performance using Metrics like R2.

## SOURCE CODE:

```
import pandas as pd.
df = pd. read_excel ("energy_data. xlsx")
print ( df .head())
x = df [["Temperature", "Humidity"]]
y = df [" Energy_Consumption"]
from sklearn. model_selection import train_
    test_split.
x_train, x_test, y_train, y_test = train_test_
        split(x, y, test_size = 0.3, random_state=
                                             42)
from sklearn ensemble import Random Forest
                       Regression
model = Random Forest Regression( n_estimators
                      =100, random_state = 42)
model. fit (x_train, y_train)
y_pred = model. predict (x_test)
print (" Predicted : ", y_pred)
print ("Actual : ", y_test values)
from sklearn, metrices import r2_score,
                    mean_squared_error
r2 = r2_score (y_test, y_pred)
mse = mean_squared_error (y_test, y_pred)
print ("R2 Score : ", r2)
print ("MSE: ", mse)
```

OUTPUT:

Predicted : [30.5   29.62]

Actual : [32.  34]

R2 score : -9.71719.99999

MSE : 10.7171999997

A. Aj
27/03/2026